

Casos de Uso (Detallado)

Descripciones extensas de flujos, excepciones y reglas de negocio

Versión	1.0
Fecha	10 de mayo de 2026
Autor	Bryan Quispe
Alcance	MVP en desarrollo: Flujos principales y alternos detallados

Índice

1. Propósito y alcance	2
2. Convenciones	2
3. CU-01: Registrarse en la plataforma	3
4. CU-02: Iniciar sesión	5
5. CU-03: Registrar una mascota	6
6. CU-04: Ver lista de mascotas	8
7. CU-05: Ver detalle de mascota y modelo 3D	9
8. CU-06: Expiración de sesión por inactividad	11
9. CU-07: Subir modelo 3D (Admin)	12
10.CU-08: Editar modelo 3D (Usuario)	14
11.Anexos	16
12.Referencias	17

1. Propósito y alcance

Documento de casos de uso con trazabilidad a requisitos funcionales (RF) y reglas de negocio (RN). Un caso de uso describe la interacción del usuario con el sistema para alcanzar un objetivo concreto dentro de una aplicación web en desarrollo. Incluye precondiciones, flujo normal, flujos alternos, excepciones, reglas de negocio, datos de entrada/salida y criterios de aceptación.

2. Convenciones

- Identificador: CU-XX
- Actor principal: usuario o rol humano que inicia la interacción con el sistema.
- Actores secundarios: otros usuarios o roles humanos involucrados.
- Tracing: (RF-YY) indica requisito funcional relacionado.

3. CU-01: Registrarse en la plataforma

Identificador	CU-01
Nombre	Registrarse en la plataforma
Actor principal	Usuario no autenticado
Actores secundarios	Sistema interno
Precondiciones	El usuario accede al formulario de registro.
Requisitos relacionados	RF-01

Datos de entrada: email, contraseña, confirmación contraseña, nombre opcional.

Datos de salida: respuesta REST con estado 201 y objeto usuario (sin contraseña) o error con mensajes de validación.

Flujo normal (exhaustivo)

1. Usuario completa formulario y envía datos.
2. Frontend valida formato básico (email, longitud contraseña) y envía petición POST /api/auth/register.
3. Backend valida reglas de negocio: email único, contraseña cumple política (≥ 8 caracteres, opcional: mayúsculas/números).
4. Si validación OK, backend hasha contraseña y crea registro en tabla users.
5. Backend retorna 201 Created con id y email.
6. Frontend muestra mensaje de éxito y redirige a login.

Flujos alternos y excepciones

- FA-01: Email ya registrado. Backend responde 409 Conflict con mensaje "Email ya existe". Fin del caso, se sugiere recuperar contraseña o usar otro email.
- FA-02: Contraseña no cumple política. Backend responde 400 Bad Request con detalle de validación.
- FA-03: Error en BD (p.ej. timeouts). Backend responde 500 Internal Server Error; frontend muestra mensaje genérico y permite reintento.

Reglas de negocio

- RB-01: Email debe ser único en la tabla users.
- RB-02: Contraseña solicitada no debe almacenarse en texto plano.

Criterios de aceptación

Respuesta 201 y nuevo registro creado; pruebas E2E que validen flujo normal y FA-01/FA-02.

Reglas de negocio

- RB-01: El email no puede repetirse en otro registro.
- RB-02: La contraseña debe almacenarse solo como hash.
- RB-03: El sistema no debe exponer la contraseña en ninguna respuesta.

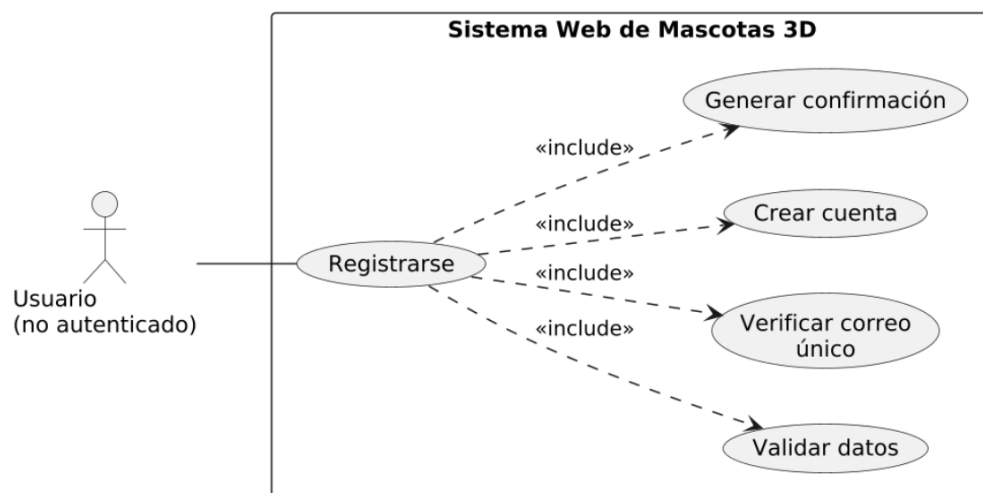


Figura 1: Diagrama conceptual: CU-01 Registrarse en la plataforma

4. CU-02: Iniciar sesión

Identificador	CU-02
Nombre	Iniciar sesión
Actor principal	Usuario no autenticado
Actores secundarios	Sistema interno
Precondiciones	Usuario registrado con credenciales válidas.
Requisitos relacionados	RF-02

Flujo normal:

1. Usuario envía email y contraseña a POST /api/auth/login.
2. Backend valida existencia de usuario y compara contraseña con Bcrypt.
3. Si coincide, backend genera JWT firmado e incluye 'userId' y 'email' y fecha de expiración.
4. Backend retorna 200 OK con token; frontend guarda token en almacenamiento seguro.

Flujos alternos

- FA-01: Credenciales inválidas ->401 Unauthorized con mensaje genérico.
- FA-02: Usuario bloqueado por intentos fallidos ->423 Locked, indicar bloqueo temporal.

Criterios de aceptación

Autenticación correcta produce JWT usable para endpoints protegidos; pruebas unitarias y de integración cubren 401 y 423.

Reglas de negocio

- RB-04: El token emitido debe contener identificador de usuario y expiración.
- RB-05: Tras varios intentos fallidos, el sistema puede bloquear temporalmente el acceso.
- RB-06: La sesión de cliente debe caducar por inactividad según la política del proyecto.

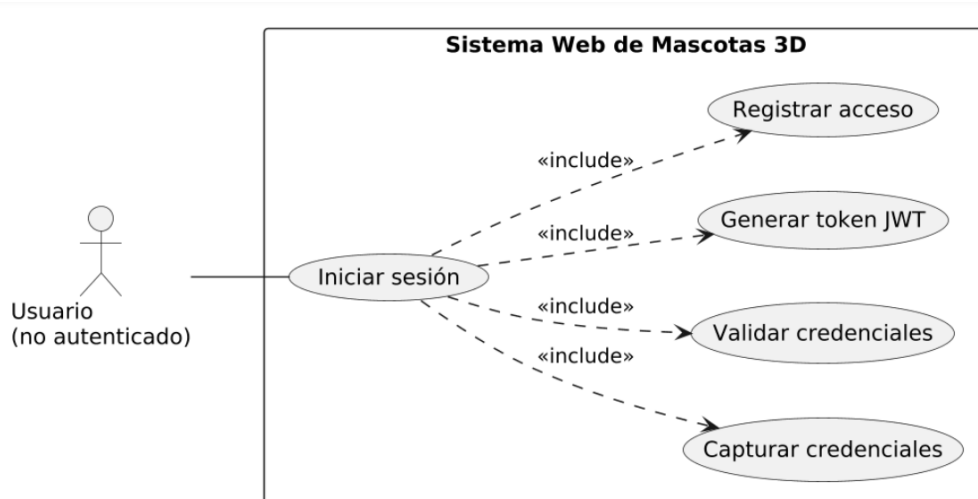


Figura 2: Diagrama conceptual: CU-02 Iniciar sesión

5. CU-03: Registrar una mascota

Identificador	CU-03
Nombre	Registrar una mascota
Actor principal	Usuario autenticado
Actores secundarios	Sistema interno
Precondiciones	Token válido, usuario autenticado.
Requisitos relacionados	RF-03, RF-06 (si incluye foto)

Flujo normal:

1. Usuario completa formulario: nombre, edad, tipo, (foto opcional).
2. Frontend valida campos y adjunta token en Authorization header.
3. Backend valida token, extrae userId y valida campos obligatorios.
4. Si hay foto, backend la procesa (comprimir/validar tipo y tamaño) y guarda en almacenamiento.
5. Backend crea registro en tabla mascotas con referencia al usuario y ruta de foto/modelo.
6. Backend retorna 201 con ID de mascota.

Flujos alternos

- FA-01: Token expirado ->401 Unauthorized, frontend redirige a login.
- FA-02: Foto más grande que 5MB ->400 Bad Request con detalle.

- FA-03: Tipo de archivo no permitido ->415 Unsupported Media Type.

Criterios de aceptación

Registro creado asociado al usuario; la foto (si aplica) es accesible mediante URL segura.

Reglas de negocio

- RB-07: Una mascota siempre pertenece a un solo usuario.
- RB-08: La foto es opcional y no debe impedir el registro básico.
- RB-09: El sistema debe validar extensión y tamaño antes de almacenar archivos.

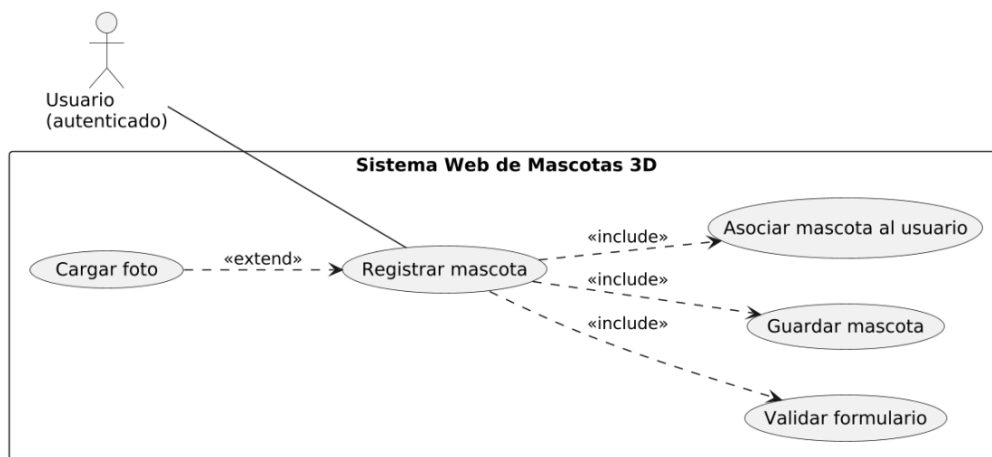


Figura 3: Diagrama conceptual: CU-03 Registrar una mascota

6. CU-04: Ver lista de mascotas

Identificador	CU-04
Nombre	Ver lista de mascotas
Actor principal	Usuario autenticado
Actores secundarios	Sistema interno
Precondiciones	Usuario autenticado con token válido.
Requisitos relacionados	RF-04

Flujo normal:

1. Frontend solicita GET /api/mascotas con Authorization header.
2. Backend valida token, consulta DB por userId, aplica paginación si procede.
3. Backend retorna lista en JSON con campos: id, nombre, edad, tipo, urlFoto, fechaCreacion.
4. Frontend renderiza la lista y permite interacción (ver detalle, editar, eliminar según permisos).

Flujos alternos

- FA-01: Ninguna mascota registrada ->retornar lista vacía y mostrar CTA para crear.
- FA-02: Error de consulta ->500 Internal Server Error y mensaje amigable al usuario.

Criterios de aceptación

La lista muestra solo mascotas del usuario; tiempos de respuesta dentro de RNF-02.

Reglas de negocio

- RB-10: Solo deben mostrarse mascotas asociadas al usuario autenticado.
- RB-11: Si no existen registros, la vista debe mostrar un estado vacío explícito.

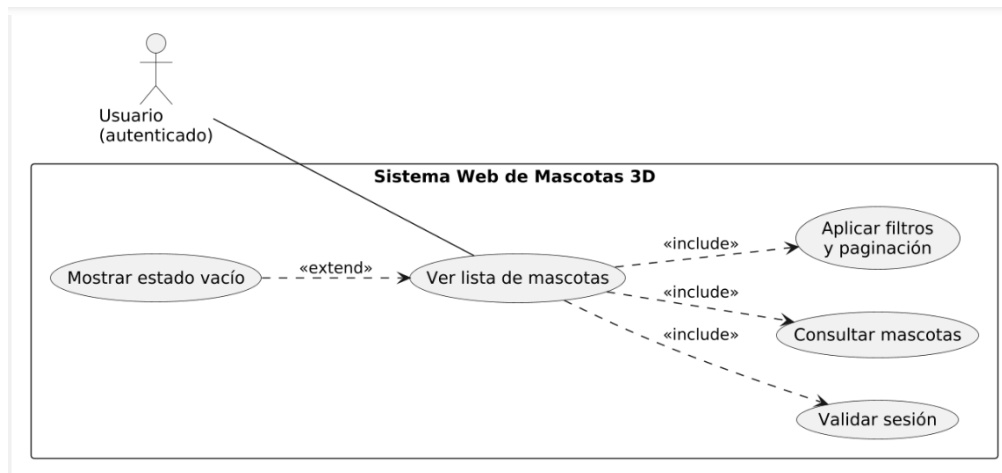


Figura 4: Diagrama conceptual: CU-04 Ver lista de mascotas

7. CU-05: Ver detalle de mascota y modelo 3D

Identificador	CU-05
Nombre	Ver detalle de mascota y modelo 3D
Actor principal	Usuario autenticado
Actores secundarios	Sistema interno
Precondiciones	Usuario autenticado y mascota existente.
Requisitos relacionados	RF-05

Flujo normal:

1. Usuario selecciona mascota en listado; frontend solicita GET /api/mascotas/id.
2. Backend valida token y que la mascota pertenezca al usuario.
3. Backend retorna detalles y la URL del modelo 3D (GLB/GLTF) asociado.
4. Frontend carga el GLB con Three.js y renderiza en canvas interactivo.

Flujos alternos

- FA-01: Mascota no encontrada o no pertenece al usuario ->404 Not Found o 403 Forbidden.
- FA-02: Modelo 3D falla al cargar ->mostrar placeholder y permitir reintento.

Criterios de aceptación

El modelo se carga correctamente en la mayoría de sesiones y permite interacción básica; tiempos dentro de RNF-02.

Reglas de negocio

- RB-12: El detalle de una mascota solo debe mostrarse si pertenece al usuario autenticado.
- RB-13: Si el modelo 3D no está disponible, la aplicación debe degradar a una vista segura sin bloquear la pantalla.

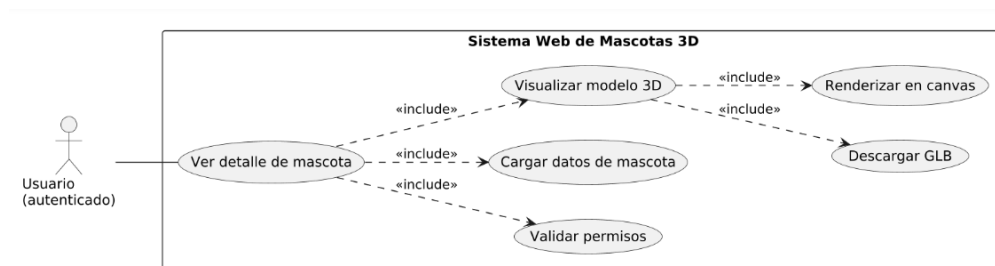


Figura 5: Diagrama conceptual: CU-05 Ver detalle de mascota y modelo 3D

8. CU-06: Expiración de sesión por inactividad

Identificador	CU-06
Nombre	Expiración de sesión por inactividad
Actor principal	Usuario autenticado
Actores secundarios	Sistema interno
Precondiciones	Usuario autenticado con una sesión activa en el cliente.
Requisitos relacionados	RNF-05

Descripción: La sesión del usuario expira automáticamente tras 10 minutos de inactividad o al cerrar la aplicación; no se modela como un requisito funcional independiente de logout manual.

Flujo normal:

1. El usuario interactúa con la aplicación y se reinicia el contador de inactividad.
2. El sistema monitorea el tiempo sin actividad del usuario.
3. Cuando se alcanza el umbral de 10 minutos, el sistema invalida la sesión local.
4. El cliente elimina token y datos temporales.
5. El usuario es redirigido a login al intentar volver a usar una ruta protegida.

Criterios de aceptación

La sesión expira a los 10 minutos de inactividad y el usuario no puede seguir navegando en rutas protegidas hasta autenticarse nuevamente.

Flujos alternos

- FA-01: El usuario vuelve a interactuar antes de los 10 minutos ->el temporizador se reinicia.
- FA-02: La aplicación se cierra ->las credenciales locales se eliminan al reiniciar la sesión del cliente.

Reglas de negocio

- RB-14: La expiración por inactividad debe ser automática.
- RB-15: El cliente no debe conservar credenciales locales después de perder la sesión.

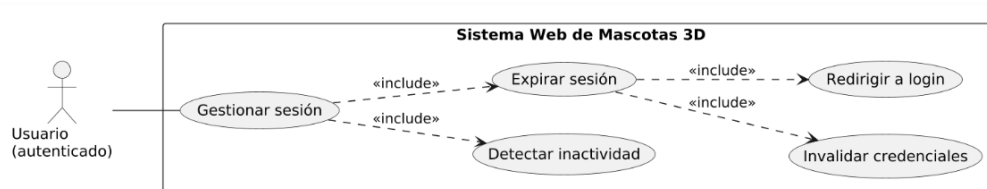


Figura 6: Diagrama conceptual: CU-06 Expiración de sesión por inactividad

9. CU-07: Subir modelo 3D (Admin)

Identificador	CU-07
Nombre	Subir modelo 3D al catálogo
Actor principal	Administrador (admin)
Actores secundarios	Sistema interno
Precondiciones	Admin autenticado con permisos de publicación.
Requisitos relacionados	RF-07

Flujo normal:

1. Admin sube archivo GLB/GLTF con metadatos (nombre, tipo, versión).
2. Backend valida formato y tamaño, guarda archivo en almacenamiento y registra metadatos en DB.
3. Backend crea versión y marca el modelo como 'publicado' si corresponde.
4. Backend retorna 201 Created con id del modelo.

Flujos alternativos

- FA-01: Formato inválido ->415 Unsupported Media Type.
- FA-02: Tamaño excede límite ->400 Bad Request.
- FA-03: Error de almacenamiento ->500 Internal Server Error.

Reglas de negocio

- RB-16: Solo el administrador puede publicar modelos 3D nuevos.
- RB-17: Cada subida válida debe generar una versión identificable.
- RB-18: El sistema debe conservar metadatos asociados al archivo.

Postcondiciones

- Modelo disponible en catálogo con versión asociada.

Criterios de aceptación

201 Created y modelo accesible vía URL; metadatos correctos.

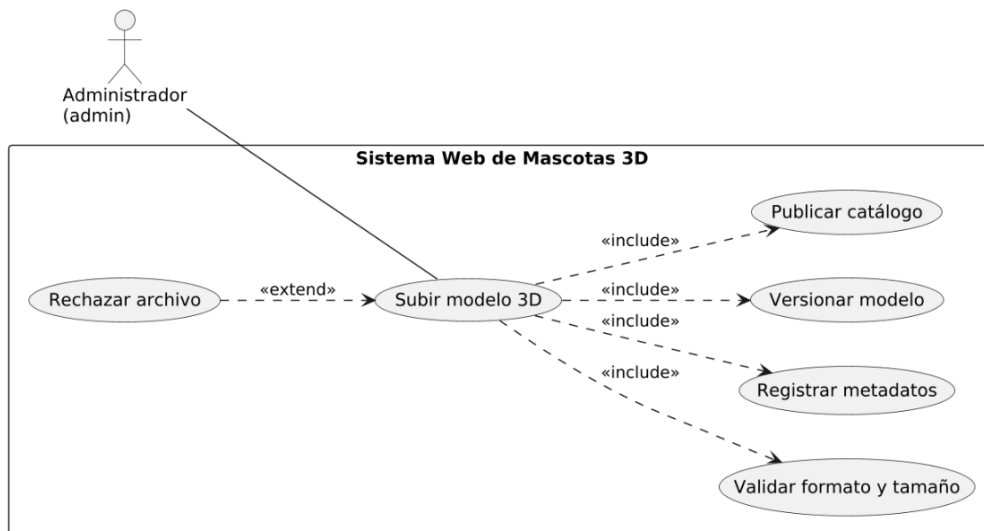


Figura 7: Diagrama conceptual: CU-07 Subir modelo 3D

10. CU-08: Editar modelo 3D (Usuario)

Identificador	CU-08
Nombre	Editar / personalizar modelo 3D
Actor principal	Usuario autenticado
Actores secundarios	Administrador
Precondiciones	Usuario autenticado; modelo disponible y con campos editables permitidos.
Requisitos relacionados	RF-07

Flujo normal:

1. El usuario selecciona el modelo 3D que desea personalizar.
2. El sistema valida permisos y determina qué campos pueden editarse.
3. El usuario modifica los campos permitidos, como nombre visible, color o metadatos asociados.
4. El sistema guarda los cambios y actualiza la versión visible del modelo.
5. Si el cambio requiere aprobación, el sistema envía la solicitud al administrador.

Flujos alternativos

- FA-01: Usuario no tiene permisos ->403 Forbidden.
- FA-02: El modelo no permite edición ->mostrar mensaje informativo y bloquear el formulario.
- FA-03: La modificación requiere aprobación ->estado 'pendiente' hasta revisión del admin.

Reglas de negocio

- RB-19: Las ediciones deben registrar qué cambió y cuándo.
- RB-20: Solo se permiten campos editables definidos por el sistema.
- RB-21: Un cambio sensible puede requerir aprobación del administrador.

Postcondiciones

- Metadatos actualizados o solicitud de revisión registrada.

Criterios de aceptación

Metadatos actualizados correctamente o solicitud registrada para revisión por admin.

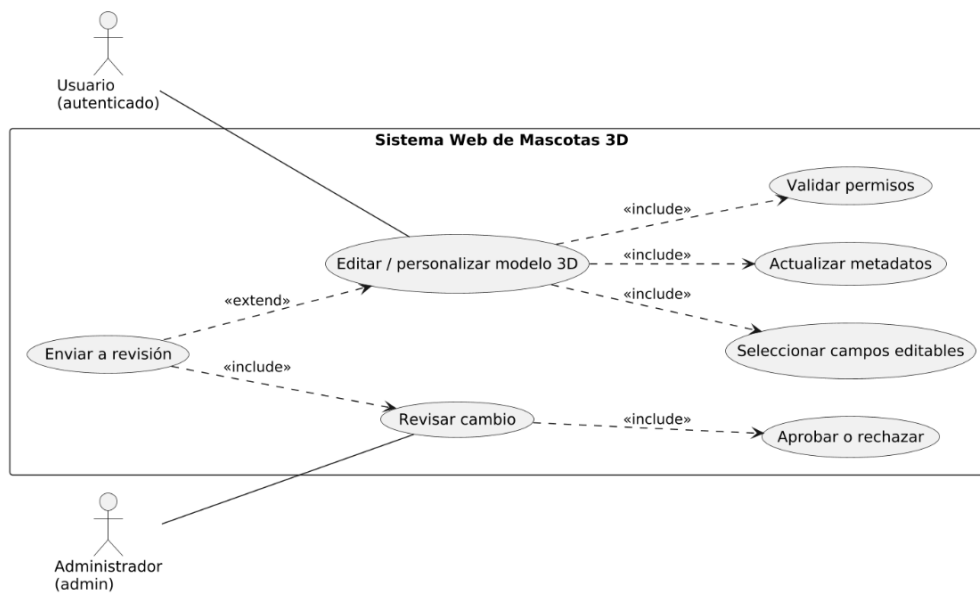


Figura 8: Diagrama conceptual: CU-08 Editar / personalizar modelo 3D

11. Anexos

Matriz de trazabilidad (ejemplo)

Requisito	Casos de uso relacionados
RF-01	CU-01
RF-03	CU-03
RNF-05	CU-06
RF-07	CU-07, CU-08

Notas de implementación

Las APIs deben devolver códigos HTTP apropiados y mensajes consistentes para facilitar validaciones en frontend y pruebas automáticas.

12. Referencias

Fuentes y plantillas consultadas para el formato y convenciones de casos de uso:

Referencias

- [1] K. Schwaber y J. Sutherland, *The Scrum Guide: The Definitive Guide to Scrum: The Rules of the Game*, The Scrum Guide, 2020. Disponible en: <https://scrumguides.org>
- [2] IEEE, *IEEE Recommended Practice for Software Requirements Specifications*, IEEE Std 830-1998.
- [3] Alistair Cockburn, *Writing Effective Use Cases*, Addison-Wesley, 2000. (Referencia para estructura y diagramación de casos de uso)